

Lab 10: Pointers and Arrays

CSE 4108

Structured Programming I Lab

Group A

Lab Tasks

1. Capital Punishment:

Write a program that reads a sentence, then capitalizes each word in it. Capitalizing means for each word, the first letter will be uppercase and all other letters will be lowercase. You can detect the beginning of a word by checking whether the previous character was either space or if the previous one (imaginary) is before the starting of the sentence. The sentence might contain digits or non alphabet characters. They will remain as they are.

Sample run:

```
Enter a sentence: toGetHer WE aRe STRong!  
Capitalized is: Together We Are Strong!
```

Note: The program should use pointers instead of integers to keep track of positions in the array.

Note: You can simplify the program by taking advantage of the fact that an array name can be used as a pointer.

2. So Close yet So Far:

Write two functions, where both would take an array and the array length as input. One function will return a pointer to the value closest to the average value of that array (function 1), while the other would return a pointer to the value farthest from average in the array (function 2). For instance, if the array is like `arr = {0, 2, 3, 7}`, average is 3. So the pointer to the closest element would point to `&arr[2]`, and the pointer to farthest would point to `&arr[3]`. Remember to take absolute distances. The declaration to functions look like below.

```
double* PtrToClosestToAvg(double *arr, int len); // function1  
double* PtrToFarthestToAvg(double *arr, int len); // function2
```

Print the Closest and Farthest values in the main function after calling the above two functions.

Sample Run:

```
Enter length of array: 4
Enter 5 elements: 0 2 3 7
Closest Value: 3
Farthest Value: 7
```

Hint: You may need to use `abs(int)` function from `<stdlib.h>` or `fabs(double)` function from `<math.h>` at some places. Or do it manually using select statements.

3. Why So Frequent!:

Write a program that takes a sentence as input from the user and prints the letter that occurs the most. The input letters will all be in lowercase. There might be punctuations and those should be ignored. You can consider that a sentence will never be larger than 50 characters.

Sample run:

```
Enter sentence: the quieter you become, the more you can hear.
Most frequent character: e
```

4. Get Rid of Em:

Write a function that takes a string and another string to hold the modification, and finally the length as its parameters and returns the length of the modified string, where all vowels have been eliminated. The declaration of the function will be as follows:

```
int getRid(const char *original, char *modified, int len);
```

Note that there are 10 vowels: 'A', 'E', 'I', 'O', 'U', 'a', 'e', 'i', 'o' and 'u'.

Sample Run:

Enter a String: Time grinds even mountains to dust.

Output: Tm grnds vn mntns t dst.